

Car Fleet Problem

Absolutely. **Car Fleet** is one of those problems where the code is short, but the **intuition is the important part**.

Let's build the idea from zero.

1. What is the question actually asking?

You have cars:

- Each car starts at some **position**
- Each car has a **speed**
- Everyone wants to reach **target**
- **A faster car cannot pass a slower car**
- If a faster car catches a slower car, they become **one fleet**
- Once they become a fleet, they move at the **slower car's speed**

You need to answer:

How many separate groups of cars will reach the target?

Example

```
target = 12
position = [10, 8, 0, 5, 3]
speed = [2, 4, 1, 1, 3]
```

Think of the road:

```
0 ---- 3 ---- 5 ---- 8 ---- 10 ----- 12
   ↑      ↑      ↑      ↑
   car    car    car    car
```

Now look at the cars.

Car at 10

```
position = 10
speed = 2
distance = 12 - 10 = 2
time = 2 / 2 = 1 hour
```

It reaches target in **1 hour**.

Car at 8

```
position = 8
speed = 4
distance = 12 - 8 = 4
time = 4 / 4 = 1 hour
```

It also reaches target in **1 hour**.

So these two cars **meet exactly at the target**.

Therefore:

```
10, 8 → one fleet
```

Car at 5

```
distance = 12 - 5 = 7
speed = 1
time = 7 / 1 = 7 hours
```

Car at 3

```
distance = 12 - 3 = 9
speed = 3

time = 9 / 3 = 3 hours
```

The car at 3 is faster.

It will catch the car at 5.

Therefore:

```
3, 5 → one fleet
```

Car at 0

```
distance = 12
speed = 1

time = 12 hours
```

Nobody behind it can catch it.

So:

```
0 → one fleet
```

Final:

```
[10,8] → fleet
[5,3] → fleet
[0] → fleet

Answer = 3
```

2. The key observation

The most important thing is:

Don't simulate the cars moving.

Instead, calculate:

```
time = (target - position) / speed
```

This tells us:

"If this car were driving alone, how much time would it need to reach the target?"

This is the key to the whole problem.

3. Why do we sort by position?

Imagine:

```
target = 12

0 ---- 3 ---- 5 ---- 8 ---- 10 ---- 12
  car    car    car    car
```

Cars can only interact with cars **in front of them**.

For example:

```
car at 3
```

can catch:

```
car at 5
car at 8
car at 10
```

But it can't catch a car at 0, because that car is behind it.

So we process cars from:

closest to target → farthest from target

That means:

```
10
8
5
3
0
```

4. Now the important intuition

For every car, calculate its arrival time.

For our example:

	Position	Speed	Time to target
	10	2	1
	8	4	1
	5	1	7
	3	3	3
	0	1	12

After sorting by position:

```
position: 10  8  5  3  0
time:    1  1  7  3 12
```

Now process from left to right in this sorted order.

First car

```
time = 1
```

Nobody is ahead of it.

So:

```
fleets = 1
```

We remember:

```
lastTime = 1
```

Second car

```
time = 1
```

Compare:

```
currentTime = 1
lastTime     = 1
```

They reach at the same time.

Therefore they become one fleet.

So:

```
fleets = 1
```

Third car

```
time = 7
```

Compare:

```
currentTime = 7
lastTime     = 1
```

This car would arrive **later**.

That means the car behind cannot catch the fleet ahead.

So it forms a **new fleet**.

```
fleets = 2
lastTime = 7
```

Fourth car

```
time = 3
```

Compare:

```
currentTime = 3
lastTime = 7
```

Here is the important part.

The car at position 3 would reach the target in only **3 hours**.

But the fleet ahead takes **7 hours**.

Therefore the car behind is faster and will eventually catch that fleet.

So:

```
3 < 7
```

means:

This car joins the fleet ahead.

No new fleet.

Fifth car

```
time = 12
```

Compare:

```
12 > 7
```

It is slower than the fleet ahead.

Therefore it cannot catch it.

New fleet:

```
fleets = 3
```

Final:

```
3 fleets
```

5. The golden rule

When processing cars from **closest to target** → **farthest from target**:

If

```
currentTime <= previousTime
```

then:

```
current car catches the fleet ahead
```

So **same fleet**.

If

```
currentTime > previousTime
```

then:

```
current car cannot catch the fleet ahead
```

So **new fleet**.

This is basically the entire problem.

6. Why does `currentTime <= previousTime` mean same fleet?

Suppose:

```
Car A:
position = 8
time = 4 hours
```

and behind it:

```
Car B:
position = 5
time = 2 hours
```

Car B would reach the target faster:

```
2 < 4
```

But B **cannot pass A**.

Therefore B must eventually catch A.

They become:

```
A + B = one fleet
```

Now suppose:

```
A = 4 hours
B = 7 hours
```

B is behind and would take longer.

So B can't catch A.

Therefore:

```
A → fleet 1
B → fleet 2
```

That's why:

```
currentTime > previousTime
```

creates a new fleet.

7. The algorithm

So our thought process becomes:

Step 1

Combine:

```
position[i]
speed[i]
```

into pairs.

Step 2

Sort cars by position in **descending order**.

Example:

```
(10,2)
(8,4)
(5,1)
(3,3)
(0,1)
```

Step 3

For every car calculate:

```
C++
time = (target - position) / speed
```

Step 4

Keep the arrival time of the **last fleet**.

Step 5

For every car:

```
if currentTime > lastTime
    new fleet
else
    joins existing fleet
```

8. C++ code

```
C++
class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {

        int n = position.size();

        // Store {position, speed}
        vector<pair<int, int>> cars;

        for (int i = 0; i < n; i++) {
            cars.push_back({position[i], speed[i]});
        }

        // Closest to target first
        sort(cars.rbegin(), cars.rend());

        int fleets = 0;

        // Time of the fleet directly ahead
        double lastTime = 0;

        for (auto &[pos, spd] : cars) {

            // Time required to reach target
            double time = (double)(target - pos) / spd;

            // This car cannot catch the fleet ahead
            if (time > lastTime) {
                fleets++;

                // This becomes the new fleet ahead
                lastTime = time;
            }

            // If time <= lastTime:
            // this car catches the fleet ahead,
            // so no new fleet is created.
        }

        return fleets;
    }
};
```

9. Dry run of the code

For:

```
C++
target = 12
position = [10,8,0,5,3]
speed    = [2,4,1,1,3]
```

After sorting:

position	speed
10	2
8	4
5	1
3	3
0	1

Now:

Car	Time	lastTime	Fleet
10	1	0	1
8	1	1	1
5	7	1	2

3	3	7	2
0	12	7	3

Answer:

3

10. Why double?

Because:

```
C++
(target - pos) / speed
```

may not be an integer.

For example:

```
target = 10
position = 3
speed = 4
```

Time:

$7 / 4 = 1.75$ hours

So we need:

```
C++
double
```

That's why:

```
C++
double time = (double)(target - pos) / spd;
```

11. A slightly deeper intuition

The clever part of this problem is that **we don't actually care about the exact collision position or collision time.**

We only care about:

Can this car catch the fleet in front before reaching the target?

And arrival time answers exactly that.

Suppose the car ahead has:

arrival time = 5

Behind car has:

arrival time = 3

It is faster overall.

So it **must catch** the car ahead somewhere before the target.

```
3 < 5
↓
catch
↓
same fleet
```

But:

arrival time = 7

means it is too slow.

```
7 > 5
↓
cannot catch
↓
new fleet
```

So the entire problem reduces to comparing **arrival times**.

12. The pattern to remember

Whenever you see:

Cars moving toward the same destination + cannot overtake + faster car joins slower car

Immediately think:

```
1. Sort by position from destination backwards
2. Calculate time to destination
3. Compare with fleet ahead
4. currentTime <= previousTime → same fleet
5. currentTime > previousTime → new fleet
```

The core intuition is:

A car behind forms a new fleet only if its individual arrival time is greater than the arrival time of the fleet in front. Otherwise, it catches that fleet.

Complexity

Sorting dominates:

```
Time:  $O(n \log n)$ 
Space:  $O(n)$ 
```

For $n = 10^5$, this is easily fast enough.

Yes. In fact, **Car Fleet is very naturally solvable using a stack**. The previous solution was already behaving like a stack, but we can make that explicit.

The important thing is that we don't actually need to store every fleet if we only care about the **arrival time of the fleet ahead**.

Stack intuition

First sort cars from **closest to target** → **farthest from target**:

```
target
↑
10 → 8 → 5 → 3 → 0
```

Calculate their times:

```
position: 10  8  5  3  0
time:      1  1  7  3  12
```

Now use a stack to store **fleet arrival times**.

Car at 10

```
time = 1
stack = [1]
```

One fleet.

Car at 8

```
time = 1
```

Compare with top:

```
1 <= 1
```

It catches the fleet ahead.

So **don't push it**.

```
stack = [1]
```

Car at 5

```
time = 7
```

Compare:

```
7 > 1
```

It cannot catch the fleet ahead.

So create a new fleet:

```
stack = [1, 7]
```

Car at 3

```
time = 3
```

Compare with top:

```
3 <= 7
```

It catches the fleet whose arrival time is 7.

Don't push.

```
stack = [1, 7]
```

Car at 0

```
time = 12
```

```
12 > 7
```

New fleet:

```
stack = [1, 7, 12]
```

Therefore:

```
stack.size() = 3
```

Stack code

```
C++
class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {
        int n = position.size();

        // {position, speed}
        vector<pair<int, int>> cars;

        for (int i = 0; i < n; i++) {
            cars.push_back({position[i], speed[i]});
        }

        // Closest to target first
        sort(cars.rbegin(), cars.rend());

        stack<double> st;

        for (auto &[pos, spd] : cars) {
            // Time required to reach target
            double time = (double)(target - pos) / spd;

            // If current car cannot catch the fleet ahead,
            // it forms a new fleet.
            if (st.empty() || time > st.top()) {
                st.push(time);
            }

            // Otherwise:
            // time <= st.top()
            // current car joins the fleet ahead.
        }

        return st.size();
    }
};
```

Why is this a stack?

Think of the stack as:

```
TOP
↓
[12] ← fleet from position 0
[ 7] ← fleet from positions 3,5
[ 1] ← fleet from positions 8,10
```

Each element represents **one fleet**.

And because we process from the front of the road toward the back, the `top()` is always the **fleet immediately ahead of the current car**.

That's why we only need:

```
C++
st.top()
```

But there is an even simpler observation

Notice what the stack code is doing:

```
C++
if (st.empty() || time > st.top())
    st.push(time);
```

We never:

- `pop()`
- access anything except `top()`

So technically, we **don't need a stack**.

We only need:

```
C++
double lastTime;
```

That's why the first solution is cleaner:

```
C++
if (time > lastTime) {
    fleets++;
    lastTime = time;
}
```

Comparison

Approach	Data structure	Complexity
Simple	<code>lastTime</code>	$O(n \log n)$, $O(n)$ for cars
Stack	<code>stack<double></code>	$O(n \log n)$, $O(n)$
Simulation	Move cars physically	Unnecessarily complicated

So **yes, stack works**, and it's a good way to understand the problem. But once you notice that the stack only ever needs its top element, you can optimize the implementation into the `lastTime` version.

The important pattern:

`current time <= stack.top()` → merge with existing fleet

`current time > stack.top()` → create new fleet.

Yes — **the reverse sorting is the most important part of the intuition**.

Let's understand exactly why.

1. First understand the road

Suppose:

```
target = 12
0 ---- 3 ---- 5 ---- 8 ---- 10 ---- 12
```

Cars are moving → **right side**, toward 12.

So:

```
10 = closest to target
8  = second closest
5  = third closest
3  = fourth closest
0  = farthest
```

Therefore, if we sort positions in **descending order**:

```
10, 8, 5, 3, 0
```

we are processing cars:

from the front of the road → toward the back of the road.

2. Why do we need to process from front to back?

Because a car can only interact with a car/fleet **in front of it**.

For example:

```
0 ---- 3 ---- 5 ---- 8 ---- 10 ---- 12
  ↑       ↑
  current fleet
```

The car at 3 can potentially catch the fleet at 5, 8, or 10.

But it cannot catch something at 0, because 0 is **behind it**.

So when we are processing the car at 3, we want the fleet in front of it to **already be processed**.

That's exactly what reverse sorting gives us.

3. Example

Take:

```
position = [10, 8, 5, 3, 0]
```

Sort descending:

```
10 → 8 → 5 → 3 → 0
```

Now process:

Car 10

Nobody is in front.

```
10 → fleet
```

Car 8

The fleet at 10 is already known.

So we can ask:

Can car 8 catch the fleet at 10?

Car 5

Now fleets in front are already known.

Ask:

Can car 5 catch the fleet ahead?

Car 3

Again, the fleet ahead is already known.

Car 0

Same thing.

So at every step:

```
current car
  ↓
[ car ] → [fleet already processed] → target
```

This is exactly what we want.

4. What happens if we sort normally?

Suppose we sort ascending:

```
0, 3, 5, 8, 10
```

Then we process:

```
0
↓
3
↓
5
↓
8
↓
10
```

But look at the first car:

```
0 ---- 3 ---- 5 ---- 8 ---- 10 ---- 12
↑
current
```

The cars that 0 needs to potentially catch are **ahead of it**:

```
3, 5, 8, 10
```

But we haven't processed them yet!

So the information we need is not available.

5. The deeper reason

The problem is essentially asking:

Can the current car catch the fleet immediately ahead of it?

Therefore, we need to know the fleet ahead **before** processing the current car.

So we process:

```
closest to target
↓
farther
↓
farther
↓
farthest
```

Which means:

```
descending position
```

6. Think of it like a queue of cars

Imagine:

```
TARGET
↓
A
B
C
D
E
```

A is closest to target.

We first determine what happens to A.

Then B asks:

"Can I catch A?"

Then C asks:

"Can I catch the fleet formed by A/B?"

Then D asks:

"Can I catch that fleet?"

So information flows:

```
A
↓
A+B
↓
A+B+C
↓
A+B+C+D
```

That's why we naturally process **front** → **back**.

7. Why does this work with the stack?

After sorting:

```
10, 8, 5, 3, 0
```

Suppose arrival times are:

```
1, 1, 7, 3, 12
```

Stack:

```
10 → time 1
stack = [1]

8 → time 1
stack = [1] // joins

5 → time 7
stack = [1, 7] // new fleet

3 → time 3
stack = [1, 7] // joins fleet 7

0 → time 12
stack = [1, 7, 12] // new fleet
```

Notice:

```
C++
st.top()
```

always represents the **fleet immediately ahead**.

That is only possible because we processed from:

closest to target → **farthest from target**.

8. One line to remember

For Car Fleet, remember this:

Sort positions in descending order because cars move toward the target, and we need to process the car in front before the car behind so that we can determine whether the behind car catches the fleet ahead.

So:

```
C++
sort(cars.rbegin(), cars.rend());
```

means:

closest to target → **farthest from target**

and that gives us the correct processing order.

ChatGPT can make mistakes. Check important info.